



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 7

Dr. Asad Mahmood

Feb. 02, 2026

Lecture Outline

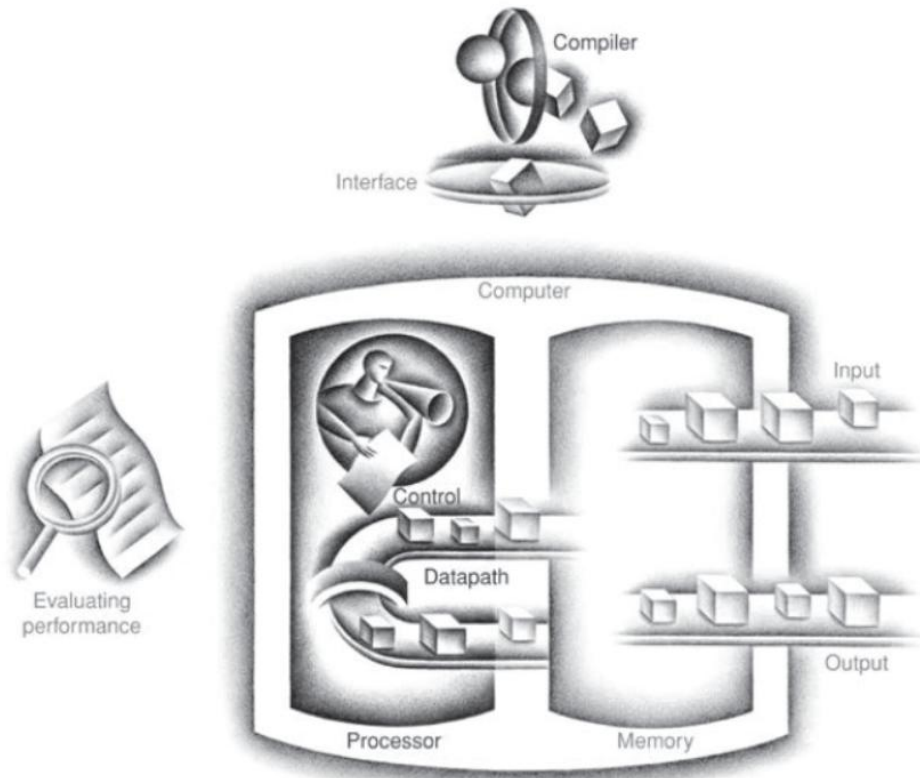
- Announcements (Assignment and Quiz 1 Day/Time/Venue)
- Outline of Previous Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
- Outline to Today's Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 -

Chapter 2

Instructions: Language of the Computer

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

- **2 Instructions: Language of the Computer**
 - [2.1 Introduction](#)
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Introduction

- To command a computer's hardware, you must speak its language.
 - The language of computer is a set of instructions called as the instruction set.
- Different computers have different instruction sets
 - But with many aspects in common
 - Examples: RISC-V, MIPS, x86 etc.
- Many computers have a very simple instruction sets
 - Simplified implementation
- Instruction Set Architecture (ISA) = Instruction Set + rules + data types, addressing modes etc.
- ISA is a consequence of the Stored-Program concept.

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - **2.2 Operations of the Computer Hardware**
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

The RISC-V Instruction Set

- Used as the example throughout the book
- Developed at UC Berkeley as open ISA
- Now managed by the RISC-V Foundation (riscv.org)
- Similar ISAs have a large share of embedded core market
 - Applications in consumer electronics, network/storage equipment, cameras, printers, ...

Arithmetic Operations

- Add and subtract, three **operands**
 - Two sources and one destination

add a, b, c // a gets b + c
- All arithmetic operations have this form
- *Design Principle 1: Simplicity favors regularity*
 - Regularity makes implementation simpler
 - Simplicity enables higher performance at lower cost

RISC-V Operands and Assemble Language

Name	Example	Comments
32 registers	x0-x31	Fast locations for data. In RISC-V, data must be in registers to perform arithmetic. Register x0 always equals 0.
2 ³⁰ memory words	Memory[0], Memory[4], ..., Memory[4,294,967,292]	Accessed only by data transfer instructions. RISC-V uses byte addresses, so sequential word accesses differ by 4. Memory holds data structures, arrays, and spilled registers.

Category	Instruction	Example	Meaning	Comments
Arithmetic	Add	add x5, x6, x7	x5 = x6 + x7	Three register operands; add
	Subtract	sub x5, x6, x7	x5 = x6 - x7	Three register operands; subtract
	Add immediate	addi x5, x6, 20	x5 = x6 + 20	Used to add constants
	Load word	lw x5, 40(x6)	x5 = Memory[x6 + 40]	Word from memory to register
Logical	And	and x5, x6, x7	x5 = x6 & x7	Three reg. operands; bit-by-bit AND
	Inclusive or	or x5, x6, x8	x5 = x6 x8	Three reg. operands; bit-by-bit OR
	Exclusive or	xor x5, x6, x9	x5 = x6 ^ x9	Three reg. operands; bit-by-bit XOR
	And immediate	andi x5, x6, 20	x5 = x6 & 20	Bit-by-bit AND reg. with constant
Data transfer	Load word	lw x5, 40(x6)	x5 = Memory[x6 + 40]	Word from memory to register
	Load word, unsigned	lwu x5, 40(x6)	x5 = Memory[x6 + 40]	Unsigned word from memory to register
	Store word	sw x5, 40(x6)	Memory[x6 + 40] = x5	Word from register to memory
	Load halfword	lh x5, 40(x6)	x5 = Memory[x6 + 40]	Halfword from memory to register
	Load halfword, unsigned	lhu x5, 40(x6)	x5 = Memory[x6 + 40]	Unsigned halfword from memory to register

Arithmetic Example

- C code:

$a = b + c;$

$d = a - e;$

- Compiled RISC-V code?

Arithmetic Example

- C code:

$$f = (g + h) - (i + j);$$

- Compiled RISC-V code?

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - **2.3 Operands of the Computer Hardware**
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

RISC-V Operands

- Three types of operand are commonly used in the RISC-V Instruction Set / Assembly Language:
 1. Register Operands
 2. Memory Operands
 3. Immediate Operands

Register Operands

- Arithmetic instructions use **register** operands
- Registers are **small-sized, fastly-accessed memory locations** very close to CPU
- **RISC-V has a 32×64 -bit register-file**
 - Use for frequently accessed data
 - **32-bit data is called a “word”**
 - 64-bit data is called a **“doubleword”**
 - 32 x 64-bit general purpose registers x0 to x31
- *Design Principle 2: Smaller is faster*
 - Larger register-file size may increase delay

RISC-V Registers

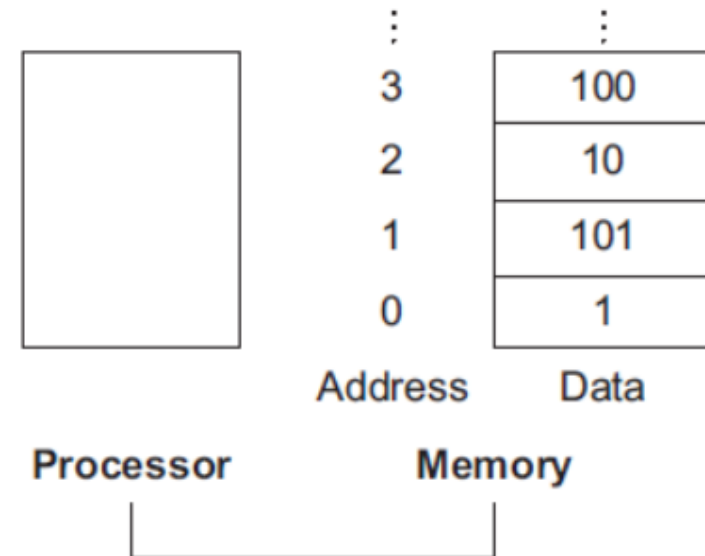
- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8: frame pointer
- x9, x18 – x27: saved registers
- x10 – x11: function arguments/results
- x12 – x17: function arguments

Register Operand Example

- C code:
 $f = (g + h) - (i + j);$
 - $f, \dots, j$ in $x19, x20, \dots, x23$
- Compiled RISC-V code?

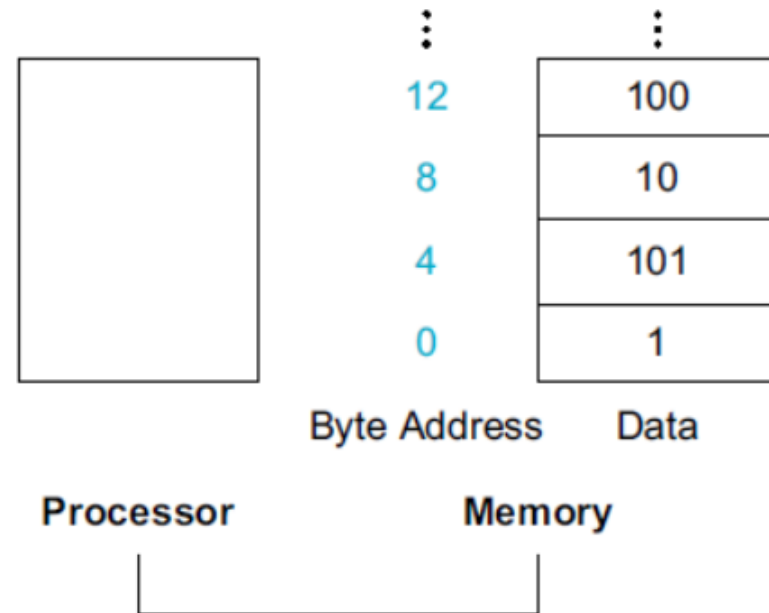
Memory Operands

- Main memory used to store more complex data-structures, e.g. Arrays, structures, dynamic data etc.
- To apply arithmetic operations
 - Data transfer instructions (load, store)
 - Load values from memory into registers
 - Store result from register to memory
- Memory is just a large, single-dimensional array, with the address acting as the index to that array, starting at 0.



Memory Operands

- Memory is byte addressed
 - Each address identifies an 8-bit byte
- RISC-V is Little Endian
 - Least-significant byte at least address of a word
 - *c.f.* Big Endian: most-significant byte at least address
- RISC-V does not require words to be aligned in memory
 - Unlike some other ISAs



Memory Operand Example

- Assume an array A of 100 words
- C code:
 $g = h + A[8];$
 - Variables g, h in registers x20 and x21 respectively
 - base address of A in register x22
- Compiled RISC-V code?
- Variant where the addition is stored in a memory location.

Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data require more instructions to be executed
- Compiler must use registers for variables as much as possible
 - Only spill to memory for less frequently used variables
 - Register optimization is important!
- Assuming 32-bit data, registers are roughly 200 times faster (0.25 vs. 50 nanoseconds) and are 10,000 times more energy efficient (0.1 vs. 1000 picoJoules) than DRAM in 2020.

Immediate Operands

- Constant data specified in an instruction

`addi x22, x22, 4`

- Make the common case fast
 - Small constants are common
 - Immediate operand avoids a load instruction
 - x0 dedicated for storing 0

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - **2.4 Signed and Unsigned Numbers**
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Binary Integers

- Computer work with numbers in their **binary format** -> **why?**
- Given an **n-bit number**
 - The corresponding decimal value?
 - Range?
 - Unsigned vs Signed
- **RISC-V has both 32-bit and 64-bit variants**
 - 32-bit/64-bit has correspondingly 32-bit/64-bit register width -> **We will focus on 32-bit RISC-V variant**

Signed Integers

- Different formats for representing signed nos.
 - Corresponding pros. and cons.?
- Given an n-bit 2's complement number
 - -ive nos. will always have a 1 in MSB -> can be considered as a 'sign' bit
 - The corresponding decimal value?
 - Range?
 - Overflow

Sign Extension

- Representing a number using more bits
 - Preserve the numeric value
- Replicate the sign bit to the left
 - c.f. unsigned values: extend with 0s
- Examples: 4-bit to 8-bit
 - $+2 \Rightarrow ?$
 - $-2 \Rightarrow ?$
- In RISC-V instruction set
 - 1b: sign-extend loaded byte
 - 1bu: zero-extend loaded byte

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - **2.5 Representing Instructions in the Computer**
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24